

BioMechAI — Kaggle PoseC3D v7 Training Notebook Manual

Technical Reference for the Training Pipeline, Environment, and Code Execution

KAGGLE GPU
EXECUTION

1. Overview and Identity of the Notebook

The **PoseC3D v7 Champion Model** was trained using the official Jupyter notebook titled:

```
biomechai-posec3d-v7-dedup-clean.ipynb
```

It is saved locally in your repository at two locations:

- **Full Run Execution with Outputs:** `defense_documentation/KAGGLE_POSEC3D_V7_TRAINING_AND_EXECUTION_OUTPUTS.ipynb`
- **Clean Source Template:** `model_training/experiments_v7_expansion/biomechai-posec3d-v7-dedup-clean.ipynb`

2. Hardware and Training Environment

COMPUTE PLATFORM Kaggle Cloud GPU Accelerated Environment	GRAPHICS CARD (GPU) NVIDIA Tesla T4 (16 GB GDDR6 VRAM) / CUDA 12.1
DEEP LEARNING FRAMEWORK PyTorch 2.1.2 + OpenMMLab MMDetection v2.19.0	VISION BACKBONE SlowOnly 3D ResNet-50 (FineGYM Limb Pretrained)

3. Mounted Input Datasets on Kaggle

The notebook ingested three pre-processed data files attached to the Kaggle runtime environment:

FILE NAME	SIZE	DESCRIPTION AND PURPOSE
<code>custom_dataset_train_v7.pkl</code>	78.4 MB	The full training pool containing 3,159 clean, deduplicated exercise clips across 7 classes.
<code>custom_dataset_val_frozen.pkl</code>	7.8 MB	The strictly frozen 444-clip held-out validation benchmark (zero data leakage across 115 unseen subjects).
<code>gym-limb_20220815-2e6e3c5c.pth</code>	126.8 MB	Official OpenMMLab FineGYM athletic limb pretrained weights for transfer learning initialization.

4. Cell-by-Cell Architecture of the Notebook (16 Cells)

The notebook consists of 16 structured cells arranged into 7 logical engineering phases:

1. **Cells 1 and 2 (Setup and Installation):** Installs `mmengine`, `mmdcv-lite`, and `mmaction2` directly from source, applying runtime compatibility patches for PyTorch 2.x arbitrary object unpickling.
2. **Cells 4 and 5 (Hardware Verification):** Runs automated hardware inspection scripts checking GPU device availability, CUDA driver version, and allocating dedicated memory buffers.
3. **Cells 6 and 7 (Experiment Settings and Hyperparameters):** Configures experiment variables, including the experiment name (`v7_dedup`), minimum joint motion threshold (`MIN_MOTION = 5.0` with `plank = 1.0` hold override), maximum fake frames allowed (`MAX_FAKE_FRAMES = 30`), and enables inverse-frequency class weighting to eliminate bias.

BioMechAI — Kaggle PoseC3D v7 Training Notebook Manual

Data Hygiene Checks, Training Loop, Evaluation, and Benchmark Results

PART 2: TRAINING & BENCHMARKS

4. **Cells 8 and 9 (Data Audit and Deduplication Engine):** Applies the MD5 hashing algorithm over all 3D pose landmark matrices, purging 987 byte-identical duplicate clips, cross-checking candidates against the validation set to eliminate 4 overlap clips, and confirming zero leakage before model initialization.
5. **Cells 10 and 11 (Training Loop — PoseC3D SlowOnly-R50):** Defines custom spatiotemporal augmentation (`RandomRotateKeypoints` within plus-or-minus 12 degrees), builds 3D limb heatmaps using Gaussian cylinder radius $\sigma = 0.6$, and trains for 18 epochs using SGD with cosine annealing learning rate scheduling.
6. **Cells 12 and 13 (Dual Benchmark Evaluation):** Runs automated validation inference using MMAction2 testing tools on both the 192 strictly unique clips and the 444 frozen baseline clips, calculating Top-1 accuracy, Macro Recall, and 95% Confidence Intervals.
7. **Cells 14 and 15 (Artifact Packaging and Export):** Extracts the best-performing model checkpoint (`best_acc_top1_epoch_10.pth`), bundles the Python configuration file, and writes all training logs and metrics to a downloadable ZIP file.

5. Training Hyperparameters Table

HYPERPARAMETER	CONFIGURED VALUE	BIOMECHANICAL / ALGORITHMIC RATIONALE
Input Modality	3D Limb Heatmaps ($\sigma = 0.6$)	Connects joints into anatomical cylinders, preventing sagittal plane self-occlusion.
Temporal Sampling	48 Frames (Stride 2)	Captures 2.5 seconds of physical movement (the natural duration of 1 full repetition).
Batch Size	16 Clips per Batch	Optimal memory footprint fitting within 16 GB GPU VRAM.
Optimizer	SGD (Momentum = 0.9)	Standard robust optimizer for 3D spatiotemporal ResNet architectures.
Learning Rate	0.01 (Cosine Decay)	Gradual cosine annealing preventing gradient explosion in early epochs.
Total Epochs	18 Epochs	Sufficient convergence without overfitting on clean training data.
Class Weighting	Inverse-Frequency Balanced	Compensates for slight sample variations across the 7 exercise classes.

6. Official Execution Results from Kaggle (Cell 13 Output)

The exact empirical evaluation logs printed directly in the notebook output:

EVALUATION TARGET	SAMPLE SIZE	TOP-1 ACCURACY	95% CONFIDENCE INTERVAL	MACRO RECALL
Best Checkpoint (Unique Clips)	192 Clips	66.15%	[59.2% to 72.5%]	63.28%
Active Backend Champion (v7)	192 Clips	63.54%	[56.5% to 70.0%]	61.39%
Frozen Benchmark (Historic)	444 Clips	53.38%	[48.7% to 58.0%]	53.15%

EVALUATION TARGET	SAMPLE SIZE	TOP-1 ACCURACY	95% CONFIDENCE INTERVAL	MACRO RECALL
FYP-I Leaked Baseline	N/A	84.00% (Invalid)	N/A	83.50%

Per-Class Performance on Clean Validation Clips:

- **Lunge:** 84.0% Recall (21 / 25 correct)
- **High Knees:** 73.7% Recall (14 / 19 correct)
- **Plank:** 67.9% Recall (19 / 28 correct)
- **Squat:** 65.6% Recall (21 / 32 correct)
- **Push-Up:** 62.5% Recall (10 / 16 correct)
- **Jumping Jack:** 51.5% Recall (17 / 33 correct)
- **Bicep Curl:** 35.9% Recall (14 / 39 correct)

7. How to Answer the Panel About the Kaggle Training (Speech Script)

SPEAK THIS TO THE PANEL:

"Respected panel, our PoseC3D v7 Champion model was trained on a Kaggle Cloud GPU environment using an NVIDIA Tesla T4 GPU with PyTorch and MMAAction2. The training notebook executed 18 epochs on our audited 3,159 clean clips using 3D connected limb heatmaps ($\sigma = 0.6$) and uniform 48-frame temporal sampling. By eliminating 987 duplicate clips and 284 noise frames, the model converged smoothly, achieving a Top-1 Accuracy of 63.54% with a Macro Recall of 61.39% on strictly unique held-out test clips. The resulting weights file, `best_acc_top1_epoch_10.pth`, is what powers our live FastAPI backend today."